

Prodhive — Backend Implementation Plan (Phases 1-6)

Repo: github.com/2303031050134-cell/prodhive | **Modules:** Prodhive_auth, Prodhive_core, Prodhive_gateway

Starting point: your existing auth/register/login, JWT gateway propagation, and basic Project/Board/ Sprint/Issue/Comment CRUD with WebSocket broadcast — all confirmed working in the repo.

Ends at: full Phase 6 of your Build Plan — Organizations, project membership + activity history, issue labels/links/subtasks/attachments, a governed Kanban workflow with filters, and backlog ranking + sprint lifecycle + analytics.

How to read this: every section says **NEW** (create the file) or **MODIFY** (change an existing file in your repo — the field names shown match the pattern already in your codebase; if your exact file differs slightly, adapt the diff to your real fields rather than overwriting blindly).

Roadmap

Phase	What it adds	Touches
1	Foundation — env/secrets, Docker Compose, CI	All 3 services
2	Organizations, invitations, email verification, password reset	Prodhive_auth
3	Project membership, settings, activity history	Prodhive_core
4	Labels, issue links, subtasks, attachments	Prodhive_core
5	Governed Kanban workflow + filters	Prodhive_core
6	Backlog ranking, sprint lifecycle, analytics (burndown/velocity/cycle time/lead time)	Prodhive_core

Phase 1: Foundation

Goal: stop hardcoding secrets/URLs, make the whole system runnable with one command, add a CI smoke check.

1a. Externalize secrets via environment variables

MODIFY `Prodhive_auth/src/main/resources/application.yml`, `Prodhive_core/src/main/resources/application.yml`, `Prodhive_gateway/src/main/resources/application.yml` — replace hardcoded values with `${VAR:default}` placeholders:

```
# Prodhive_auth/application.yml
server:
  port: ${SERVER_PORT:8081}
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5433/authdb}
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
  jpa:
    hibernate:
      ddl-auto: update
  jwt:
    secret: ${JWT_SECRET:replace-this-with-a-long-random-secret-in-prod}
    expiration-ms: ${JWT_EXPIRATION_MS:86400000}
```

Repeat the same `${VAR:default}` pattern for `Prodhive_core` (port 8082, coredb) and `Prodhive_gateway` (port 8080, plus `jwt.secret` — must match auth-service).

1b. .env files (NEW — one per service, gitignored)

```
# Prodhive_auth/.env
DB_URL=jdbc:postgresql://localhost:5433/authdb
DB_USER=postgres
DB_PASSWORD=postgres
JWT_SECRET=change-me-to-a-real-64-char-random-string
JWT_EXPIRATION_MS=86400000
```

Add `.env` to each service's `.gitignore` (your `.gitignore` files already exist — just confirm `.env` is listed).

1c. Dockerfile per service (NEW)

```
# Prodhive_auth/Dockerfile (same content in Prodhive_core, Prodhive_gateway)
FROM eclipse-temurin:21-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

1d. docker-compose.yml (NEW — repo root)

```

version: '3.8'
services:
  auth-db:
    image: postgres:16
    environment:
      POSTGRES_DB: authdb
      POSTGRES_PASSWORD: postgres
    ports: ["5433:5432"]

  core-db:
    image: postgres:16
    environment:
      POSTGRES_DB: coredb
      POSTGRES_PASSWORD: postgres
    ports: ["5434:5432"]

  prodhive-auth:
    build: ./Prodhive_auth
    depends_on: [auth-db]
    environment:
      DB_URL: jdbc:postgresql://auth-db:5432/authdb
      JWT_SECRET: ${JWT_SECRET}
    ports: ["8081:8081"]

  prodhive-core:
    build: ./Prodhive_core
    depends_on: [core-db]
    environment:
      DB_URL: jdbc:postgresql://core-db:5432/coredb
      JWT_SECRET: ${JWT_SECRET}
    ports: ["8082:8082"]

  prodhive-gateway:
    build: ./Prodhive_gateway
    depends_on: [prodhive-auth, prodhive-core]
    environment:
      JWT_SECRET: ${JWT_SECRET}
    ports: ["8080:8080"]

```

Create a single root-level `.env` with `JWT_SECRET=...` (same value everywhere) — Compose reads it automatically.

1e. Minimal CI (NEW — `.github/workflows/build.yml`)

```

name: Build & Test
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        module: [Prodhive_auth, Prodhive_core, Prodhive_gateway]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
      - name: Build and test ${matrix.module}
        run: cd ${matrix.module} && ./mvnw clean verify

```

✓ **Phase 1 done when:** `docker compose up --build` from the repo root brings up both databases and all 3 services cleanly, and a push to GitHub triggers the Actions workflow and passes.

Phase 2: Organizations & Auth Hardening

Goal: add the multi-tenancy concept your plan calls for (currently missing entirely), plus email verification and password reset. All in `Prodhive_auth`.

2a. New entity: Organization

NEW `entity/Organization.java`

```
package com.prodhive_auth.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Organization {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) private String name;
    @Column(nullable = false, unique = true) private String slug; // url-safe, e.g. "acme-corp"

    @Column(nullable = false) private Long ownerId; // User.id who created it
    @Column(nullable = false, updatable = false) private Instant createdAt = Instant.now();
}
```

2b. New entity: OrganizationMember

NEW `entity/OrganizationMember.java`

```
package com.prodhive_auth.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
@Table(uniqueConstraints = @UniqueConstraint(columnNames = {"organization_id", "user_id"}))
public class OrganizationMember {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "organization_id", nullable = false)
    private Organization organization;

    @ManyToOne @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private OrgRole role = OrgRole.MEMBER; // OWNER, ADMIN, MEMBER

    @Column(nullable = false, updatable = false) private Instant joinedAt = Instant.now();
}
```

```
// entity/OrgRole.java
package com.prodhive_auth.entity;
public enum OrgRole { OWNER, ADMIN, MEMBER }
```

Your existing `User.role` (ADMIN/PROJECT_MANAGER/MEMBER) becomes a **platform-level** role (mostly relevant for your own admin tooling). Real day-to-day permissions now flow through `OrganizationMember.role` and, in Phase 3, `ProjectMember.role`. Don't delete the existing field — just stop treating it as the source of truth for project/org permissions.

2c. New entity: Invitation

NEW `entity/Invitation.java`

```

package com.prodhive_auth.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Invitation {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "organization_id", nullable = false)
    private Organization organization;

    @Column(nullable = false) private String email;
    @Column(nullable = false, unique = true) private String token;
    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private OrgRole role = OrgRole.MEMBER;

    @Column(nullable = false) private Long invitedBy;
    @Column(nullable = false) private Instant expiresAt;
    private Instant acceptedAt; // null until accepted

    @Column(nullable = false, updatable = false) private Instant createdAt = Instant.now();
}

```

2d. Repositories (NEW)

```

public interface OrganizationRepository extends JpaRepository<Organization, Long> {
    Optional<Organization> findBySlug(String slug);
}

public interface OrganizationMemberRepository extends JpaRepository<OrganizationMember, Long> {
    List<OrganizationMember> findByOrganizationId(Long orgId);
    List<OrganizationMember> findByUserId(Long userId);
    Optional<OrganizationMember> findByOrganizationIdAndUserId(Long orgId, Long userId);
}

public interface InvitationRepository extends JpaRepository<Invitation, Long> {
    Optional<Invitation> findByToken(String token);
}

```

2e. OrganizationService (NEW)

```

package com.prodhive_auth.service;

import com.prodhive_auth.entity.*;
import com.prodhive_auth.repository.*;
import org.springframework.stereotype.Service;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.UUID;

@Service
public class OrganizationService {

    private final OrganizationRepository orgRepository;
    private final OrganizationMemberRepository memberRepository;
    private final InvitationRepository invitationRepository;
    private final UserRepository userRepository;

    public OrganizationService(OrganizationRepository orgRepository, OrganizationMemberRepository memberRepository,
                              InvitationRepository invitationRepository, UserRepository userRepository) {
        this.orgRepository = orgRepository;
        this.memberRepository = memberRepository;
        this.invitationRepository = invitationRepository;
        this.userRepository = userRepository;
    }

    public Organization create(String name, String slug, Long ownerId) {
        Organization org = new Organization();
        org.setName(name);
        org.setSlug(slug);
        org.setOwnerId(ownerId);
        Organization saved = orgRepository.save(org);

        OrganizationMember member = new OrganizationMember();
        member.setOrganization(saved);
        member.setUser(userRepository.findById(ownerId).orElseThrow());
        member.setRole(OrgRole.OWNER);
        memberRepository.save(member);

        return saved;
    }

    public List<OrganizationMember> listMembers(Long orgId) {
        return memberRepository.findByOrganizationId(orgId);
    }

    public Invitation invite(Long orgId, String email, OrgRole role, Long invitedBy) {
        Organization org = orgRepository.findById(orgId).orElseThrow();
        Invitation invitation = new Invitation();
        invitation.setOrganization(org);
        invitation.setEmail(email);
        invitation.setToken(UUID.randomUUID().toString());
        invitation.setRole(role);
        invitation.setInvitedBy(invitedBy);
        invitation.setExpiresAt(Instant.now().plus(7, ChronoUnit.DAYS));
        return invitationRepository.save(invitation);

        // NOTE: no real email is sent here — see 2f note below.
    }

    public OrganizationMember acceptInvitation(String token, Long acceptingUserId) {
        Invitation invitation = invitationRepository.findByToken(token)
            .orElseThrow(() -> new IllegalArgumentException("Invalid invitation token"));
        if (invitation.getAcceptedAt() != null) throw new IllegalArgumentException("Invitation already used");
        if (invitation.getExpiresAt().isBefore(Instant.now())) throw new IllegalArgumentException("Invitation expired");

        invitation.setAcceptedAt(Instant.now());
        invitationRepository.save(invitation);

        OrganizationMember member = new OrganizationMember();
        member.setOrganization(invitation.getOrganization());
        member.setUser(userRepository.findById(acceptingUserId).orElseThrow());
        member.setRole(invitation.getRole());
        return memberRepository.save(member);
    }
}

```

No SMTP/email service is implemented here. Sending real invitation/verification emails needs an email provider (SendGrid, AWS SES, or plain SMTP via Spring Mail) — that's genuinely extra infrastructure, not a code gap you can close in an afternoon. For your timeline: `invite()` and the verification flow below both generate a real token; for the demo, return the token/link directly in the API response (or log it server-side) instead of emailing it, and name "transactional email delivery" explicitly as future work in your report. This is a completely normal and defensible scoping decision.

2f. OrganizationController (NEW)

```

package com.prodhive_auth.controller;

import com.prodhive_auth.entity.*;
import com.prodhive_auth.security.CurrentUser;
import com.prodhive_auth.service.OrganizationService;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/auth/organizations")
public class OrganizationController {

    private final OrganizationService orgService;
    private final CurrentUser currentUser;

    public OrganizationController(OrganizationService orgService, CurrentUser currentUser) {
        this.orgService = orgService;
        this.currentUser = currentUser;
    }

    @PostMapping
    public ResponseEntity<Organization> create(@RequestBody CreateOrgRequest req, HttpServletRequest http) {
        Long userId = currentUser.getUserId(http);
        return ResponseEntity.ok(orgService.create(req.name(), req.slug(), userId));
    }

    @GetMapping("/{id}/members")
    public ResponseEntity<List<OrganizationMember>> members(@PathVariable Long id) {
        return ResponseEntity.ok(orgService.listMembers(id));
    }

    @PostMapping("/{id}/invitations")
    public ResponseEntity<Invitation> invite(@PathVariable Long id, @RequestBody InviteRequest req,
                                             HttpServletRequest http) {
        Long userId = currentUser.getUserId(http);
        return ResponseEntity.ok(orgService.invite(id, req.email(), OrgRole.valueOf(req.role()), userId));
    }

    @PostMapping("/invitations/{token}/accept")
    public ResponseEntity<OrganizationMember> accept(@PathVariable String token, HttpServletRequest http) {
        Long userId = currentUser.getUserId(http);
        return ResponseEntity.ok(orgService.acceptInvitation(token, userId));
    }
}

// dto/CreateOrgRequest.java
record CreateOrgRequest(String name, String slug) {}
// dto/InviteRequest.java
record InviteRequest(String email, String role) {}

```

You'll need a `CurrentUser` component in `Prodhive_auth` similar to the one already in `Prodhive_core` (reads `x-User-Id` header set by the gateway) — copy that pattern over if it isn't already present in the auth service.

2g. Email verification

MODIFY `entity/User.java` — add:

```

@Column(nullable = false)
private boolean emailVerified = false;

```

NEW `entity/VerificationToken.java`

```

package com.prodhive_auth.entity;
import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class VerificationToken {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false, unique = true) private String token;
    @Column(nullable = false) private Long userId;
    @Column(nullable = false) private Instant expiresAt;
}

```

MODIFY `service/AuthService.java` — in `register()`, after saving the user, generate and store a token (same UUID pattern as invitations), and return it in the response for the demo (again: real delivery = future work). Add two new methods:

```

public void verifyEmail(String token) {
    VerificationToken vt = verificationTokenRepository.findByToken(token)
        .orElseThrow(() -> new IllegalArgumentException("Invalid verification token"));
    if (vt.getExpiresAt().isBefore(Instant.now())) throw new IllegalArgumentException("Token expired");

    User user = userRepository.findById(vt.getUserId()).orElseThrow();
    user.setEmailVerified(true);
    userRepository.save(user);
    verificationTokenRepository.delete(vt);
}

```

Add `GET /api/auth/verify?token=...` to `AuthController` calling this method.

2h. Password reset

NEW `entity/PasswordResetToken.java` — identical shape to `VerificationToken`.

MODIFY `service/AuthService.java` — add:

```

public void requestPasswordReset(String email) {
    User user = userRepository.findByEmail(email).orElse(null);
    if (user == null) return; // don't leak whether the email exists

    String token = UUID.randomUUID().toString();
    PasswordResetToken prt = new PasswordResetToken();
    prt.setToken(token);
    prt.setUserId(user.getId());
    prt.setExpiresAt(Instant.now().plus(1, ChronoUnit.HOURS));
    passwordResetTokenRepository.save(prt);
    // TODO: email the reset link containing `token` — logged for now
    System.out.println("Password reset token for " + email + ": " + token);
}

public void resetPassword(String token, String newPassword) {
    PasswordResetToken prt = passwordResetTokenRepository.findByToken(token)
        .orElseThrow(() -> new IllegalArgumentException("Invalid reset token"));
    if (prt.getExpiresAt().isBefore(Instant.now())) throw new IllegalArgumentException("Token expired");

    User user = userRepository.findById(prt.getUserId()).orElseThrow();
    user.setPasswordHash(passwordEncoder.encode(newPassword));
    userRepository.save(user);
    passwordResetTokenRepository.delete(prt);
}

```

MODIFY `controller/AuthController.java` — add:

```

@PostMapping("/forgot-password")
public ResponseEntity<Void> forgotPassword(@RequestBody ForgotPasswordRequest req) {
    authService.requestPasswordReset(req.email());
    return ResponseEntity.ok().build();
}

@PostMapping("/reset-password")
public ResponseEntity<Void> resetPassword(@RequestBody ResetPasswordRequest req) {
    authService.resetPassword(req.token(), req.newPassword());
    return ResponseEntity.ok().build();
}

// dto records: ForgotPasswordRequest(String email), ResetPasswordRequest(String token, String newPassword)

```

2i. JWT — add organizationId claim

MODIFY `security/JwtUtil.java` — add an `orgId` claim, and update `AuthService.login / register` to pass the user's org (their first/only org for now — multi-org switching is a reasonable Phase 13+ stretch, not needed for Phase 6):

```

public String generateToken(String email, String role, Long userId, Long orgId) {
    return Jwts.builder()
        .subject(email)
        .claim("role", role)
        .claim("userId", userId)
        .claim("orgId", orgId)
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis() + expirationMs))
        .signWith(key())
        .compact();
}

```

MODIFY `Prodhive_gateway/.../filter/JwtAuthFilter.java` — propagate it as another header:

```

.header("X-Org-Id", String.valueOf(claims.get("orgId")))

```

✓ **Phase 2 done when:** a user can register, create an org, invite a second user by email (token returned in the response), that user accepts the invite and shows up in `GET /api/auth/organizations/{id}/members` , and password reset round-trips correctly (request → use token → login with new password).

Phase 3: Projects — Membership, Settings, Activity History

Goal: projects belong to an org, have real members (not just an owner), and generate an activity feed. All in `Prodhive_core`.

3a. Link Project to Organization

MODIFY `entity/Project.java` — add:

```
@Column(nullable = false)
private Long organizationId; // references Organization.id in Prodhive_auth — no FK, cross-service

@Column(nullable = false)
private boolean archived = false;
```

MODIFY `dto/ProjectRequest.java` — add `organizationId` to the record, and **MODIFY** `service/ProjectService.create()` to set it from the request (validated against the `X-Org-Id` header from the gateway, not trusted blindly from the request body).

3b. New entity: ProjectMember

```
package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
@Table(uniqueConstraints = @UniqueConstraint(columnNames = {"project_id", "user_id"}))
public class ProjectMember {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "project_id", nullable = false)
    private Project project;

    @Column(nullable = false) private Long userId; // references User.id in Prodhive_auth

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private ProjectRole role = ProjectRole.MEMBER; // PM, MEMBER

    @Column(nullable = false, updatable = false) private Instant joinedAt = Instant.now();
}
```

```
// entity/ProjectRole.java
package com.prodhive_core.entity;
public enum ProjectRole { PM, MEMBER }
```

3c. Repository + Service (NEW)

```
public interface ProjectMemberRepository extends JpaRepository<ProjectMember, Long> {
    List<ProjectMember> findByProjectId(Long projectId);
    Optional<ProjectMember> findByProjectIdAndUserId(Long projectId, Long userId);
    void deleteByProjectIdAndUserId(Long projectId, Long userId);
}
```

```

package com.prodhive_core.service;

import com.prodhive_core.entity.*;
import com.prodhive_core.repository.ProjectMemberRepository;
import com.prodhive_core.repository.ProjectRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProjectMemberService {

    private final ProjectMemberRepository memberRepository;
    private final ProjectRepository projectRepository;

    public ProjectMemberService(ProjectMemberRepository memberRepository, ProjectRepository projectRepository) {
        this.memberRepository = memberRepository;
        this.projectRepository = projectRepository;
    }

    public ProjectMember addMember(Long projectId, Long userId, ProjectRole role) {
        Project project = projectRepository.findById(projectId).orElseThrow();
        ProjectMember member = new ProjectMember();
        member.setProject(project);
        member.setUserId(userId);
        member.setRole(role);
        return memberRepository.save(member);
    }

    public List<ProjectMember> listMembers(Long projectId) {
        return memberRepository.findByProjectId(projectId);
    }

    public void removeMember(Long projectId, Long userId) {
        memberRepository.deleteByProjectIdAndUserId(projectId, userId);
    }
}

```

3d. ProjectMemberController (NEW)

```

package com.prodhive_core.controller;

import com.prodhive_core.entity.*;
import com.prodhive_core.service.ProjectMemberService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/core/projects/{projectId}/members")
public class ProjectMemberController {

    private final ProjectMemberService memberService;

    public ProjectMemberController(ProjectMemberService memberService) {
        this.memberService = memberService;
    }

    @PostMapping
    public ResponseEntity<ProjectMember> add(@PathVariable Long projectId, @RequestBody AddMemberRequest req) {
        return ResponseEntity.ok(memberService.addMember(projectId, req.userId(), ProjectRole.valueOf(req.role())));
    }

    @GetMapping
    public ResponseEntity<List<ProjectMember>> list(@PathVariable Long projectId) {
        return ResponseEntity.ok(memberService.listMembers(projectId));
    }

    @DeleteMapping("/{userId}")
    public ResponseEntity<Void> remove(@PathVariable Long projectId, @PathVariable Long userId) {
        memberService.removeMember(projectId, userId);
        return ResponseEntity.noContent().build();
    }
}

// dto: record AddMemberRequest(Long userId, String role) {}

```

3e. Project settings — rename & archive

MODIFY `service/ProjectService.java` — add:

```

public Project update(Long id, String name, String description) {
    Project project = findById(id);
    if (name != null) project.setName(name);
    if (description != null) project.setDescription(description);
    return projectRepository.save(project);
}

public Project archive(Long id) {
    Project project = findById(id);
    project.setArchived(true);
    return projectRepository.save(project);
}

```

MODIFY `controller/ProjectController.java` — add matching `PATCH /api/core/projects/{id}` and `PATCH /api/core/projects/{id}/archive` endpoints, role-checked the same way `delete` already is.

3f. Activity history — generic across entities

NEW `entity/ActivityLog.java`

```

package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class ActivityLog {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) private Long projectId;
    @Column(nullable = false) private String entityType; // "ISSUE", "SPRINT", "BOARD", "PROJECT"
    @Column(nullable = false) private Long entityId;
    @Column(nullable = false) private String action; // "CREATED", "STATUS_CHANGED", "COMMENTED", etc.
    @Column(nullable = false) private Long actorId;
    @Column(length = 1000) private String detail; // free-text, e.g. "TODO -> IN_PROGRESS"

    @Column(nullable = false, updatable = false) private Instant createdAt = Instant.now();
}

```

```

public interface ActivityLogRepository extends JpaRepository<ActivityLog, Long> {
    List<ActivityLog> findByProjectIdOrderByCreatedAtDesc(Long projectId);
}

```

```

package com.prodhive_core.service;

import com.prodhive_core.entity.ActivityLog;
import com.prodhive_core.repository.ActivityLogRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ActivityLogService {

    private final ActivityLogRepository repository;

    public ActivityLogService(ActivityLogRepository repository) {
        this.repository = repository;
    }

    public void log(Long projectId, String entityType, Long entityId, String action, Long actorId, String detail) {
        ActivityLog entry = new ActivityLog();
        entry.setProjectId(projectId);
        entry.setEntityType(entityType);
        entry.setEntityId(entityId);
        entry.setAction(action);
        entry.setActorId(actorId);
        entry.setDetail(detail);
        repository.save(entry);
    }

    public List<ActivityLog> getProjectActivity(Long projectId) {
        return repository.findByProjectIdOrderByCreatedAtDesc(projectId);
    }
}

```

MODIFY `service/IssueService.java`, `SprintService.java`, `ProjectService.java` — inject `ActivityLogService` and call `.log(...)` at the end of each create/update method. Example in `IssueService.updateStatus` (you'll expand this further in Phase 5 anyway):

```
activityLogService.log(issue.getBoard().getProject().getId(), "ISSUE", issue.getId(),
    "STATUS_CHANGED", changedBy, oldStatus + " -> " + newStatus);
```

NEW endpoint in `ProjectController.java` :

```
@GetMapping("/{id}/activity")
public ResponseEntity<List<ActivityLog>> activity(@PathVariable Long id) {
    return ResponseEntity.ok(activityLogService.getProjectActivity(id));
}
```

✓ **Phase 3 done when:** creating a project requires a valid org, you can add/remove a project member and list them, archiving a project sets the flag without deleting data, and `GET /api/core/projects/{id}/activity` shows a real feed after creating/moving a few issues.

Phase 4: Issues — Labels, Links, Subtasks, Attachments

Goal: round out the Issue model to match what your plan calls for beyond type/status/priority/assignee/comments (already done).

4a. Labels

```
// entity/Label.java
package com.prodhive_core.entity;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Label {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false) private String name;
    @Column(nullable = false) private String color; // hex, e.g. "#0B3D91"
    @ManyToOne @JoinColumn(name = "project_id", nullable = false)
    private Project project;
}
```

MODIFY `entity/Issue.java` — add a many-to-many:

```
@ManyToMany
@JoinTable(name = "issue_labels",
    joinColumns = @JoinColumn(name = "issue_id"),
    inverseJoinColumns = @JoinColumn(name = "label_id"))
private Set<Label> labels = new HashSet<>();
```

```
public interface LabelRepository extends JpaRepository<Label, Long> {
    List<Label> findByProjectId(Long projectId);
}
```

```
// service/LabelService.java — straightforward CRUD, same pattern as ProjectService
// controller/LabelController.java — @RequestMapping("/api/core/projects/{projectId}/labels")
// POST (create), GET (list), DELETE /{id}

// New endpoints on IssueController:
@PostMapping("/{id}/labels/{labelId}")
public ResponseEntity<Issue> addLabel(@PathVariable Long id, @PathVariable Long labelId) { ... }

@DeleteMapping("/{id}/labels/{labelId}")
public ResponseEntity<Issue> removeLabel(@PathVariable Long id, @PathVariable Long labelId) { ... }
```

4b. Issue links (blocks / relates to / duplicates)

```
// entity/IssueLink.java
package com.prodhive_core.entity;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class IssueLink {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "source_issue_id", nullable = false)
    private Issue sourceIssue;

    @ManyToOne @JoinColumn(name = "target_issue_id", nullable = false)
    private Issue targetIssue;

    @Enumerated(EnumType.STRING) @Column(nullable = false)
    private LinkType linkType; // BLOCKS, RELATES_TO, DUPLICATES
}

// entity/LinkType.java
public enum LinkType { BLOCKS, RELATES_TO, DUPLICATES }
```

```
public interface IssueLinkRepository extends JpaRepository<IssueLink, Long> {
    List<IssueLink> findBySourceIssueIdOrTargetIssueId(Long sourceId, Long targetId);
}
```

```
// New endpoints on IssueController:
@PostMapping("/{id}/links")
public ResponseEntity<IssueLink> link(@PathVariable Long id, @RequestBody LinkRequest req) { ... }
// dto: record LinkRequest(Long targetIssueId, String linkType) {}

@GetMapping("/{id}/links")
public ResponseEntity<List<IssueLink>> listLinks(@PathVariable Long id) { ... }
```

4c. Subtasks

MODIFY `entity/Issue.java` — add a self-referencing parent:

```
@ManyToOne @JoinColumn(name = "parent_issue_id")
private Issue parentIssue; // null for top-level issues
```

```
// repository/IssueRepository.java — add:
List<Issue> findByParentIssueId(Long parentIssueId);
```

```
// New endpoints on IssueController:
@PostMapping("/{id}/subtasks")
public ResponseEntity<Issue> createSubtask(@PathVariable Long id, @RequestBody IssueRequest req,
                                           HttpServletRequest http) {
    Issue parent = issueService.findById(id);
    Issue subtask = issueService.create(req, currentUser.getUserId(http));
    subtask.setParentIssue(parent);
    return ResponseEntity.ok(issueService.save(subtask)); // add a plain save() passthrough in IssueService if missing
}

@GetMapping("/{id}/subtasks")
public ResponseEntity<List<Issue>> subtasks(@PathVariable Long id) {
    return ResponseEntity.ok(issueRepository.findByParentIssueId(id));
}
```

4d. Attachments (metadata + simple local-disk upload)

```
// entity/Attachment.java
package com.prodhive_core.entity;
import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class Attachment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "issue_id", nullable = false)
    private Issue issue;

    @Column(nullable = false) private String fileName;
    @Column(nullable = false) private String fileUrl; // path/URL to fetch it
    @Column(nullable = false) private Long uploadedBy;
    @Column(nullable = false, updatable = false) private Instant uploadedAt = Instant.now();
}
```

```
// controller/AttachmentController.java
package com.prodhive_core.controller;

import com.prodhive_core.entity.Attachment;
import com.prodhive_core.security.CurrentUser;
import com.prodhive_core.service.AttachmentService;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import java.util.List;

@RestController
@RequestMapping("/api/core/issues/{issueId}/attachments")
public class AttachmentController {

    private final AttachmentService attachmentService;
    private final CurrentUser currentUser;

    public AttachmentController(AttachmentService attachmentService, CurrentUser currentUser) {
        this.attachmentService = attachmentService;
        this.currentUser = currentUser;
    }

    @PostMapping
    public ResponseEntity<Attachment> upload(@PathVariable Long issueId, @RequestParam("file") MultipartFile file,
                                              HttpServletRequest http) {
        Long userId = currentUser.getUserId(http);
        return ResponseEntity.ok(attachmentService.store(issueId, file, userId));
    }

    @GetMapping
    public ResponseEntity<List<Attachment>> list(@PathVariable Long issueId) {
        return ResponseEntity.ok(attachmentService.findByIssue(issueId));
    }
}
```

```
// service/AttachmentService.java - writes to a local uploads/ folder (swap for S3 later if needed)
package com.prodhive_core.service;

import com.prodhive_core.entity.Attachment;
import com.prodhive_core.entity.Issue;
import com.prodhive_core.repository.AttachmentRepository;
import com.prodhive_core.repository.IssueRepository;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.UUID;

@Service
public class AttachmentService {

    private final AttachmentRepository attachmentRepository;
    private final IssueRepository issueRepository;
    private final Path uploadDir = Paths.get("uploads");

    public AttachmentService(AttachmentRepository attachmentRepository, IssueRepository issueRepository) {
        this.attachmentRepository = attachmentRepository;
        this.issueRepository = issueRepository;
    }

    public Attachment store(Long issueId, MultipartFile file, Long userId) {
        try {
            Files.createDirectories(uploadDir);
            String storedName = UUID.randomUUID() + "-" + file.getOriginalFilename();
            Files.copy(file.getInputStream(), uploadDir.resolve(storedName));

            Issue issue = issueRepository.findById(issueId).orElseThrow();
            Attachment attachment = new Attachment();
            attachment.setIssue(issue);
            attachment.setFileName(file.getOriginalFilename());
            attachment.setFileUrl("/uploads/" + storedName);
            attachment.setUploadedBy(userId);
            return attachmentRepository.save(attachment);
        } catch (IOException e) {
            throw new RuntimeException("Failed to store file", e);
        }
    }

    public List<Attachment> findByIssue(Long issueId) {
        return attachmentRepository.findByIssueId(issueId);
    }
}
```

Local-disk storage is fine for a demo but won't survive a container restart in the Docker Compose setup from Phase 1 (the `uploads/` folder isn't a mounted volume). Either add a Docker volume for it, or note in your report that production would use S3/GCS — both are legitimate, just be explicit about which one you did.

✓ **Phase 4 done when:** you can attach a label to an issue and filter by it (filters land in Phase 5), link two issues as BLOCKS, create a subtask under a parent issue, and upload/list a file attachment on an issue.

Phase 5: Kanban Board — Governed Workflow & Filters

Goal: stop allowing any-status-to-any-status, add an audit trail, and let the board/backlog be filtered server-side.

5a. Expand the status enum

MODIFY `entity/IssueStatus.java` — currently `TODO`, `IN_PROGRESS`, `DONE` . Expand to:

```
package com.prodhive_core.entity;
public enum IssueStatus { BACKLOG, TODO, IN_PROGRESS, IN_REVIEW, DONE, REOPENED }
```

Existing rows in `coredb` with `TODO` / `IN_PROGRESS` / `DONE` remain valid values under the new enum — no data migration needed, you're only adding values.

5b. WorkflowService (NEW)

```
package com.prodhive_core.service;

import com.prodhive_core.entity.IssueStatus;
import com.prodhive_core.exception.InvalidTransitionException;
import org.springframework.stereotype.Service;
import java.util.*;

@Service
public class WorkflowService {

    private static final Map<IssueStatus, Set<IssueStatus>> ALLOWED_TRANSITIONS = Map.of(
        IssueStatus.BACKLOG,      EnumSet.of(IssueStatus.TODO),
        IssueStatus.TODO,         EnumSet.of(IssueStatus.IN_PROGRESS, IssueStatus.BACKLOG),
        IssueStatus.IN_PROGRESS,  EnumSet.of(IssueStatus.IN_REVIEW, IssueStatus.TODO),
        IssueStatus.IN_REVIEW,    EnumSet.of(IssueStatus.DONE, IssueStatus.IN_PROGRESS),
        IssueStatus.DONE,         EnumSet.of(IssueStatus.REOPENED),
        IssueStatus.REOPENED,     EnumSet.of(IssueStatus.IN_PROGRESS)
    );

    public void validateTransition(IssueStatus from, IssueStatus to) {
        if (from == to) return;
        Set<IssueStatus> allowed = ALLOWED_TRANSITIONS.getOrDefault(from, Set.of());
        if (!allowed.contains(to)) {
            throw new InvalidTransitionException("Cannot move issue from " + from + " to " + to);
        }
    }
}
```

```
// exception/InvalidTransitionException.java
package com.prodhive_core.exception;
public class InvalidTransitionException extends RuntimeException {
    public InvalidTransitionException(String message) { super(message); }
}
```

MODIFY `exception/GlobalExceptionHandler.java` — add:

```
@ExceptionHandler(InvalidTransitionException.class)
public ResponseEntity<String> handleInvalidTransition(InvalidTransitionException e) {
    return ResponseEntity.status(HttpStatus.CONFLICT).body(e.getMessage());
}
```

5c. IssueStatusHistory (NEW — also feeds Phase 6 analytics)

```

package com.prodhive_core.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.Instant;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
public class IssueStatusHistory {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne @JoinColumn(name = "issue_id", nullable = false)
    private Issue issue;

    @Enumerated(EnumType.STRING) private IssueStatus fromStatus;
    @Enumerated(EnumType.STRING) private IssueStatus toStatus;
    @Column(nullable = false) private Long changedBy;
    @Column(nullable = false, updatable = false) private Instant changedAt = Instant.now();
}

public interface IssueStatusHistoryRepository extends JpaRepository<IssueStatusHistory, Long> {
    List<IssueStatusHistory> findByIdOrderByChangedAtAsc(Long issueId);
    List<IssueStatusHistory> findByIssue_Board_IdAndChangedAtBetween(Long boardId, Instant from, Instant to);
}

```

5d. Refactor IssueService.updateStatus

MODIFY `service/IssueService.java` — inject `WorkflowService`, `IssueStatusHistoryRepository`, and `ActivityLogService` (from Phase 3) into the constructor, then replace the existing method:

```

public Issue updateStatus(Long id, String statusStr, Long changedBy) {
    Issue issue = findById(id);
    IssueStatus oldStatus = issue.getStatus();
    IssueStatus newStatus = IssueStatus.valueOf(statusStr);

    workflowService.validateTransition(oldStatus, newStatus);

    IssueStatusHistory history = new IssueStatusHistory();
    history.setIssue(issue);
    history.setFromStatus(oldStatus);
    history.setToStatus(newStatus);
    history.setChangedBy(changedBy);
    historyRepository.save(history);

    issue.setStatus(newStatus);
    issue.setUpdatedAt(Instant.now());
    Issue saved = issueRepository.save(issue);

    messagingTemplate.convertAndSend("/topic/board/" + issue.getBoard().getId(), saved);
    activityLogService.log(issue.getBoard().getProject().getId(), "ISSUE", issue.getId(),
        "STATUS_CHANGED", changedBy, oldStatus + " -> " + newStatus);

    return saved;
}

```

MODIFY `controller/IssueController.java` — pass the acting user through:

```

@PatchMapping("/{id}/status")
public ResponseEntity<Issue> updateStatus(@PathVariable Long id, @RequestBody StatusUpdateRequest req,
                                          HttpServletRequest http) {
    Long userId = currentUser.getUserId(http);
    return ResponseEntity.ok(issueService.updateStatus(id, req.status(), userId));
}

@GetMapping("/{id}/history")
public ResponseEntity<List<IssueStatusHistory>> history(@PathVariable Long id) {
    return ResponseEntity.ok(historyRepository.findByIdOrderByChangedAtAsc(id));
}

```

5e. Server-side filters for the board/backlog

MODIFY `repository/IssueRepository.java` — add derived query methods (simplest approach, no need for full JPA Specifications at this scale):

```
public interface IssueRepository extends JpaRepository<Issue, Long> {
    List<Issue> findByBoardId(Long boardId);
    List<Issue> findByBoardIdAndSprintId(Long boardId, Long sprintId);
    List<Issue> findByBoardIdAndSprintIdIsNull(Long boardId); // backlog
    List<Issue> findByBoardIdAndAssigneeId(Long boardId, Long assigneeId);
    List<Issue> findByBoardIdAndType(Long boardId, IssueType type);
    List<Issue> findByBoardIdAndPriority(Long boardId, IssuePriority priority);
    Optional<Issue> findByIssueKey(String issueKey); // needed if you also adopt Phase-6-style keys later
}
```

MODIFY `controller/IssueController.java` — extend the list endpoint to branch on whichever optional query params are present:

```
@GetMapping
public ResponseEntity<List<Issue>> list(@RequestParam Long boardId,
                                       @RequestParam(required = false) Long sprintId,
                                       @RequestParam(required = false) Long assigneeId,
                                       @RequestParam(required = false) String type,
                                       @RequestParam(required = false) String priority,
                                       @RequestParam(required = false) Boolean backlogOnly) {
    if (Boolean.TRUE.equals(backlogOnly)) return ResponseEntity.ok(issueRepository.findByBoardIdAndSprintIdIsNull(boardId));
    if (sprintId != null) return ResponseEntity.ok(issueRepository.findByBoardIdAndSprintId(boardId, sprintId));
    if (assigneeId != null) return ResponseEntity.ok(issueRepository.findByBoardIdAndAssigneeId(boardId, assigneeId));
    if (type != null) return ResponseEntity.ok(issueRepository.findByBoardIdAndType(boardId, IssueType.valueOf(type)));
    if (priority != null) return ResponseEntity.ok(issueRepository.findByBoardIdAndPriority(boardId, IssuePriority.valueOf(priority)));
    return ResponseEntity.ok(issueRepository.findByBoardId(boardId));
}
```

This branches on one filter at a time, which is enough for your frontend's single-select filter chips. If you later want combined filters (type *and* priority *and* assignee together), switch to JPA Specifications or a query-by-example builder — flag that as a stretch goal, not required for Phase 5/6 completion.

✓ **Phase 5 done when:** dragging a card to a disallowed column returns 409 and the history endpoint shows every valid transition with who made it, and the issue list endpoint correctly filters by each supported query param individually.

Phase 6: Backlog Ranking, Sprint Lifecycle & Analytics

Goal: a real backlog order, sprint start/complete with rollover, and the four analytics your plan names: burndown, velocity, cycle time, lead time.

6a. Backlog ranking

MODIFY `entity/Issue.java` — add:

```
@Column(nullable = false)
private Double rank = 0.0; // LexoRank-lite: reorder by placing new rank between neighbors
```

MODIFY `controller/IssueController.java` — add a reorder endpoint:

```
@PatchMapping("/{id}/rank")
public ResponseEntity<Issue> updateRank(@PathVariable Long id, @RequestBody RankUpdateRequest req) {
    return ResponseEntity.ok(issueService.updateRank(id, req.newRank()));
}
// dto: record RankUpdateRequest(Double newRank) {}
```

MODIFY `service/IssueService.java` — add:

```
public Issue updateRank(Long id, Double newRank) {
    Issue issue = findById(id);
    issue.setRank(newRank);
    return issueRepository.save(issue);
}
```

Frontend computes `newRank` as the midpoint between the two neighboring issues' ranks when an item is dropped between them (e.g. dropped between rank 1.0 and 2.0 → new rank 1.5). This avoids re-numbering the whole list on every reorder — the same trick Jira itself uses (LexoRank), simplified to plain doubles for your scale.

MODIFY `repository/IssueRepository.java` — add ordering to the backlog query:

```
List<Issue> findByBoardIdAndSprintIdIsNullOrderByRankAsc(Long boardId);
```

6b. Sprint lifecycle

MODIFY `entity/Sprint.java` — confirm/add a status field (add if not already present):

```
@Enumerated(EnumType.STRING) @Column(nullable = false)
private SprintStatus status = SprintStatus.PLANNED;
// entity/SprintStatus.java (separate file, public)
// package com.prodhive_core.entity;
// public enum SprintStatus { PLANNED, ACTIVE, COMPLETED }
```

MODIFY `service/SprintService.java` — add:

```
public Sprint start(Long id) {
    Sprint sprint = findById(id);
    sprint.setStatus(SprintStatus.ACTIVE);
    return sprintRepository.save(sprint);
}

public Sprint complete(Long id) {
    Sprint sprint = findById(id);
    sprint.setStatus(SprintStatus.COMPLETED);
    sprintRepository.save(sprint);

    // roll incomplete issues back to the backlog
    List<Issue> issues = issueRepository.findBySprintId(id);
    for (Issue issue : issues) {
        if (issue.getStatus() != IssueStatus.DONE) {
            issue.setSprint(null);
            issueRepository.save(issue);
        }
    }
    return sprint;
}
```

(Inject `IssueRepository` into `SprintService` 's constructor if it isn't already there.)

MODIFY `controller/SprintController.java` — add:

```

@PatchMapping("/{id}/start")
public ResponseEntity<Sprint> start(@PathVariable Long id) {
    return ResponseEntity.ok(sprintService.start(id));
}

@PatchMapping("/{id}/complete")
public ResponseEntity<Sprint> complete(@PathVariable Long id) {
    return ResponseEntity.ok(sprintService.complete(id));
}

```

6c. Analytics — AnalyticsController + AnalyticsService (NEW)

```

package com.prodhive_core.service;

import com.prodhive_core.entity.*;
import com.prodhive_core.repository.*;
import org.springframework.stereotype.Service;
import java.time.*;
import java.time.temporal.ChronoUnit;
import java.util.*;
import java.util.stream.Collectors;

@Service
public class AnalyticsService {

    private final IssueRepository issueRepository;
    private final IssueStatusHistoryRepository historyRepository;
    private final SprintRepository sprintRepository;

    public AnalyticsService(IssueRepository issueRepository, IssueStatusHistoryRepository historyRepository,
        SprintRepository sprintRepository) {
        this.issueRepository = issueRepository;
        this.historyRepository = historyRepository;
        this.sprintRepository = sprintRepository;
    }

    // 1. Burndown: remaining (not-DONE) issue count per day of the sprint
    public List<Map<String, Object>> burndown(Long sprintId) {
        Sprint sprint = sprintRepository.findById(sprintId).orElseThrow();
        List<Issue> issues = issueRepository.findById(sprintId);
        List<Map<String, Object>> result = new ArrayList<>();

        LocalDate day = sprint.getStartDate();
        while (!day.isAfter(sprint.getEndDate())) {
            Instant cutoff = day.plusDays(1).atStartOfDay(ZoneOffset.UTC).toInstant();
            long remaining = issues.stream()
                .filter(issue -> !wasDoneBefore(issue, cutoff))
                .count();
            result.add(Map.of("date", day.toString(), "remaining", remaining));
            day = day.plusDays(1);
        }
        return result;
    }

    private boolean wasDoneBefore(Issue issue, Instant cutoff) {
        return historyRepository.findByIdOrderByChangedAtAsc(issue.getId()).stream()
            .anyMatch(h -> h.getToStatus() == IssueStatus.DONE && h.getChangedAt().isBefore(cutoff));
    }

    // 2. Velocity: issues completed per sprint, last N sprints on a board
    public List<Map<String, Object>> velocity(Long boardId, int lastNSprints) {
        List<Sprint> sprints = sprintRepository.findByBoardId(boardId).stream()
            .filter(s -> s.getStatus() == SprintStatus.COMPLETED)
            .sorted(Comparator.comparing(Sprint::getEndDate).reversed())
            .limit(lastNSprints)
            .collect(Collectors.toList());

        List<Map<String, Object>> result = new ArrayList<>();
        for (Sprint sprint : sprints) {
            long completed = issueRepository.findById(sprint.getId()).stream()
                .filter(i -> i.getStatus() == IssueStatus.DONE)
                .count();
            result.add(Map.of("sprint", sprint.getName(), "completed", completed));
        }
        Collections.reverse(result); // oldest first for charting
        return result;
    }

    // 3. Cycle time: avg hours from IN_PROGRESS -> DONE per issue on a board
    public double avgCycleTimeHours(Long boardId) {
        List<Issue> issues = issueRepository.findByBoardId(boardId);
        List<Double> cycleHours = new ArrayList<>();

        for (Issue issue : issues) {
            var history = historyRepository.findByIdOrderByChangedAtAsc(issue.getId());
            Instant startedAt = history.stream()

```

```

        .filter(h -> h.getToStatus() == IssueStatus.IN_PROGRESS)
        .map(IssueStatusHistory::getChangedAt).findFirst().orElse(null);
Instant doneAt = history.stream()
    .filter(h -> h.getToStatus() == IssueStatus.DONE)
    .map(IssueStatusHistory::getChangedAt)
    .reduce((first, second) -> second).orElse(null); // last DONE transition
if (startedAt != null && doneAt != null && doneAt.isAfter(startedAt)) {
    cycleHours.add((double) ChronoUnit.HOURS.between(startedAt, doneAt));
}
}
return cycleHours.isEmpty() ? 0.0 : cycleHours.stream().mapToDouble(Double::doubleValue).average().orElse(0.0);
}

// 4. Lead time: avg hours from issue creation -> DONE per issue on a board
public double avgLeadTimeHours(Long boardId) {
    List<Issue> issues = issueRepository.findByBoardId(boardId);
    List<Double> leadHours = new ArrayList<>();

    for (Issue issue : issues) {
        var history = historyRepository.findByIssueIdOrderByChangedAtAsc(issue.getId());
        Instant doneAt = history.stream()
            .filter(h -> h.getToStatus() == IssueStatus.DONE)
            .map(IssueStatusHistory::getChangedAt)
            .reduce((first, second) -> second).orElse(null);
        if (doneAt != null) {
            leadHours.add((double) ChronoUnit.HOURS.between(issue.getCreatedAt(), doneAt));
        }
    }
    return leadHours.isEmpty() ? 0.0 : leadHours.stream().mapToDouble(Double::doubleValue).average().orElse(0.0);
}
}

```

```

// controller/AnalyticsController.java
package com.prodhive_core.controller;

import com.prodhive_core.service.AnalyticsService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/core/boards/{boardId}/analytics")
public class AnalyticsController {

    private final AnalyticsService analyticsService;

    public AnalyticsController(AnalyticsService analyticsService) {
        this.analyticsService = analyticsService;
    }

    @GetMapping("/burndown")
    public ResponseEntity<List<Map<String, Object>>> burndown(@PathVariable Long boardId,
                                                             @RequestParam Long sprintId) {
        return ResponseEntity.ok(analyticsService.burndown(sprintId));
    }

    @GetMapping("/velocity")
    public ResponseEntity<List<Map<String, Object>>> velocity(@PathVariable Long boardId,
                                                             @RequestParam(defaultValue = "5") int last) {
        return ResponseEntity.ok(analyticsService.velocity(boardId, last));
    }

    @GetMapping("/cycle-time")
    public ResponseEntity<Map<String, Double>> cycleTime(@PathVariable Long boardId) {
        return ResponseEntity.ok(Map.of("avgHours", analyticsService.avgCycleTimeHours(boardId)));
    }

    @GetMapping("/lead-time")
    public ResponseEntity<Map<String, Double>> leadTime(@PathVariable Long boardId) {
        return ResponseEntity.ok(Map.of("avgHours", analyticsService.avgLeadTimeHours(boardId)));
    }
}

```

This computes everything on-demand from `IssueStatusHistory` rather than maintaining separate aggregate tables — simplest correct approach at your data scale. If board history grows large in a real deployment, you'd pre-aggregate nightly instead; not a concern for a student project's data volume, but worth naming as a scaling note in your report.

✔ **Phase 6 done when:** reordering backlog issues persists the new rank, starting then completing a sprint moves incomplete issues back to the backlog automatically, and all four analytics endpoints return sensible numbers after you've run a full sprint cycle through the workflow end to end.

Summary — New Files by Service

Service	New files
Prodhive_auth	Organization, OrganizationMember, OrgRole, Invitation, VerificationToken, PasswordResetToken (entities) · matching repositories · OrganizationService · OrganizationController
Prodhive_core	ProjectMember, ProjectRole, ActivityLog, Label, IssueLink, LinkType, Attachment, IssueStatusHistory, SprintStatus, InvalidTransitionException (entities/exceptions) · matching repositories · ProjectMemberService, ActivityLogService, LabelService, AttachmentService, WorkflowService, AnalyticsService · ProjectMemberController, LabelController, AttachmentController, AnalyticsController
Prodhive_gateway	No new files — one header line added to the existing filter
Repo root	docker-compose.yml, .env, .github/workflows/build.yml

This closes out Phase 6 of your Build Plan. Phases 7–15 (notifications, search, GitHub integration, PR reviews, webhooks, advanced features, testing, deployment) are not covered here — bring those back as a separate request once Phases 1–6 are working end to end, since GitHub integration in particular depends on the issue-key convention this guide doesn't introduce (your current `Issue` entity has no human-readable key — add one, same pattern as `PROJ-15`, before starting PR linkage).